

What is MongoDB ?

MongoDB is a NoSQL database that follows document-oriented model. It stores data in a flexible Json-like documents, allowing for varied and complex data structures.

Features of MongoDB:

1. **Document Model** : unlike tradition databases that stores data in form of tables and rows, MongoDB uses Document-oriented model meaning it stores data in BSON (binary Json) Documents This format is more expressive and allows for nested and hierarchical data.
2. **Scalability**: MongoDB is designed to scale horizontally, which supports sharding, which distributes data across multiple machines, making it easy to scale out as your data grows.
3. Shema flexibility: MongoDB is schema-less meaning you don't have to define your data upfront.

SQL vs NoSQL

- **SQL**
 - Type of relational database
 - Have less flexibility
 - Schema is predefined and needs complex queries for alteration
 - Data stored in rows and columns format
 - Uses Structured query language
 - E.g. MySQL, MySQL, oracle, Postgres
- **NoSQL**
 - Non-relational database
 - High flexibility, designed for high performance and scalability.
 - Can store various types of data structures – documents, graphs, key-values

- usually follows CAP theorem (consistency, availability, partition tolerance)
- data stored as collections
- each NoSQL has its own query language (MQL- MongoDB query language in case of MongoDB).
- E.g. MongoDB (collections BSON), Cassandra(key-values), neo4j(graphs)

MongoDB Tutorial

- Followed the below playlist for installation and full MongoDB learning
- URL - [MongoDB Playlist - learnt all mongodb from this playlist](#)

How Data Is Structured In MongoDB

- MongoDB stores data in form of collections (tables in sql).
- Each Database can have n number of collections
- Each collection has data stored as Documents (rows in sql)
- Documents represent single set of record of the data.
- Every document have different set of properties defined for eg. A document from blog post collection with have data like so:

```
{
  Title: "first blog",
  Author: "Hoshi",
  Description: "this is my first blog"
  _id : Object (sa08hdg239dh238s...) // - id is generated by mongodb for each
                                     // document uniquely
}
```

- MongoDB can store nested documents meaning document within document like so :

```
{
  Title: "first blog",
  Author: {
    Name: "Hoshi",
    Email: "hoshi@gmail.com"
  }
}
```

```
    Description: "this is my first blog",
    Tag: ['informative', 'funny']
    _id : Object (sa08hdg239dh238s...) // - id is generated by mongodb for each
                                         // document uniquely
  }
```

Installation

Install MongoDB compass and mongoddb shell for the hands on practice!

<https://www.mongodb.com/try/download/compass>

use the above link to download compass GUI (exe or msi) and MongoDB shell for shell commands

do note that MongoDB compass often comes with integrated mongosh terminal for running shell commands you can use this terminal or can install mongoddb shell and use it via any other terminal in your pc like cmd use mongosh command for activating mongo shell.

MongoDB Commands

Generic commands (Basics)

- Show dbs or show databases
 - Lists all available databases
- Show collections
 - Lists all available collections in a database (current selected)
- Use db name
 - It allows to select / switch a Database and perform CRUD on the selected db. Also, if db doesn't exist it will create a new db with given name.

Collection Commands

- db.createCollection("myCollection")
 - creates a collection named "mycollection"
- db.mycollection.drop()
 - will drop a collection

Adding Data – (Create)

- db.collection name.insertOne({ name: "hoshi", age:29});
 - will add a single document with the provided fields and _id. In the db collections.
- db.collection name.insertMany([
 { name: "kookie", age: 27 },
 { name: "jimin", age: 29 }
]);
 - takes input as array of objects and insert those multiple entries in the db collection.

Finding Data - (Read)

➔ **Find()** :

- db.collection name.find()
 - returns everything from db
- db.collection name.find({name:"Hoshi"})
 - conditional find() returns the entries (documents) with name = hoshi
- db.collection name.find({}, {name:1,age:1})

- the second argument in `find()` specifies which fields to return (specified with numeric 1), useful when only few fields from the document are needed and not the whole object.
- `db.collection_name.findOne( {name: "Josh"})`
 - Return a single document that matches the query.

Method Chaining and Sorting

Method chaining in mongodb refers to the practice of calling multiple methods in a single, continuous line of code, where each method operates as the output of previous one. This approach is possible because many mongodb shell commands (methods) returns a cursor or same object, allowing you to chain additional methods.

Methods Often Used in Chaining

1. `find()`:

- **Purpose:** Retrieves documents from the collection that match the given query.
- **Returns:** A cursor object that points to the documents found.
- **Example:** `db.myCollection.find({ age: { $gt: 25 } })`
- **Explanation:** This finds all documents where the age field is greater than 25.

2. `sort()`:

- **Purpose:** Sorts the documents based on a specified field. Sorts in descending order if (-1) and ascending order if (0).
- **Returns:** The same cursor object, but now ordered based on the sort criteria.
- **Example:** `.sort({ age: -1 })`
- **Explanation:** This sorts the resulting documents by the age field in descending order (-1 for descending, 1 for ascending).

3. **limit():**

- **Purpose:** Limits the number of documents returned.
- **Returns:** The same cursor object, but now limited to the specified number of documents.
- **Example:** `.limit(5)`
- **Explanation:** This limits the result set to 5 documents.

4. **project():**

- **Purpose:** Specifies which fields to include or exclude in the returned documents.
- **Example:**
`db.myCollection.find({ age: { $gt: 25 } }).project({ name: 1, age: 1, _id: 0 })`
- **Explanation:** This returns only the name and age fields, excluding the `_id` field.

5. **skip():**

- **Purpose:** Skips a specified number of documents in the result set.
- **Example:**
`db.myCollection.find({}).sort({ age: 1 }).skip(10)`
- **Explanation:** This skips the first 10 documents in the sorted result set.

6. **count():**

- **Purpose:** Counts the number of documents that match the query.
- **Example:**
`db.myCollection.find({ age: { $gt: 25 } }).count()`

- **Explanation:** This counts the number of documents where the age field is greater than 25

Example of method chaining

Let's consider an example where we want to find documents in a collection, sort them and then the number of results returned:

```
» db.myCollection.find({ age: 25 }) .sort({ age: -1 }).limit(5)
```

Nested documents

In MongoDB, nested documents are documents that contain other documents or arrays as values within their fields. This allows MongoDB to represent complex data structures in a single document, making it very flexible for modeling real-world data.

Nested documents in mdb helps read performance as we only have to make a single query to get all the information rather than querying multiple collections

Structure of Nested Documents

A nested document is simply a document embedded within another document. Here's a basic example:

```
{  
  "name": "John Doe",  
  "age": 30,  
  "address": {  
    "street": "123 Main St",  
    "city": "New York",
```

```
"zip": "10001"
},
"hobbies": ["reading", "travelling", "swimming"]
}
```

Explanation

- **name and age:** Simple key-value pairs.
- **address:** A nested document, containing its own fields (street, city, and zip).
- **hobbies:** An array of strings, which is also a form of nesting, though with simple data types.

Benefits of Nested Documents

1. **Data Grouping:** Nested documents allow you to group related data together within a single document. For example, the address fields are grouped together under a single key.
2. **Denormalization:** Unlike relational databases, where you might use multiple tables to store related data, MongoDB allows you to store related data together in one document. This can improve read performance by reducing the need for joins.
3. **Flexibility:** Nested documents can have varying structures, making MongoDB adaptable to different types of data and use cases.

Querying Nested Documents

MongoDB provides various ways to query nested documents:

1. Accessing Nested Fields:

- To query the city field within the address document:

```
db.users.find({ "address.city": "New York" })
```

2. Updating Nested Fields:

- To update the zip code in the address document:

```
db.users.updateOne( { "name": "John Doe" }, { $set: {  
  "address.zip": "10002" } })
```

3. Querying Arrays in Nested Documents:

- To find documents where hobbies include "reading":

```
db.users.find({ hobbies: "reading" })
```

Considerations

- **Document Size:** Each document in MongoDB has a size limit of 16MB. If your nested documents or arrays are large, you need to ensure they don't exceed this limit.
- **Indexing:** You can create indexes on nested fields, which can improve query performance. However, complex nested structures may require careful indexing strategies to optimize performance.

Operators and Complex Queries

MongoDB provides a rich set of query operators that allow you to perform complex queries on your data. These operators can be used to filter, project, update, or manipulate data within your MongoDB collections. By combining these operators, you can write powerful queries to interact with your MongoDB collections effectively. Here's a brief overview of some commonly used operators

Comparison Operators

Comparison operators are used to filter documents based on the values of specific fields.

1. **\$gt (Greater Than):**

- **Usage:** Finds documents where the value of a field is greater than a specified value.
- **Example:** Find all documents where age is greater than 25.

```
db.users.find({ age: { $gt: 25 } })
```

2. **\$lt (Less Than):**

- **Usage:** Finds documents where the value of a field is less than a specified value.
- **Example:** Find all documents where price is less than 100

```
db.products.find({ price: { $lt: 100 } })
```

3. **\$gte (Greater Than or Equal To):**

- **Usage:** Finds documents where the value of a field is greater than or equal to a specified value.
- **Example:** Find all documents where rating is 4.5 or higher

```
db.reviews.find({ rating: { $gte: 4.5 } })
```

4. **\$lte (Less Than or Equal To):**

- **Usage:** Finds documents where the value of a field is less than or equal to a specified value.
- **Example:** Find all documents where quantity is 10 or less.

```
db.inventory.find({ quantity: { $lte: 10 } })
```

5. **\$eq (Equal):**

- **Usage:** Finds documents where the value of a field is equal to a specified value.
- **Example:** Find all documents where status is "active".

```
db.users.find({ status: { $eq: "active" } })
```

- Or you can simply use

```
db.users.find({ status: "active" })
```

6. **\$ne (Not Equal):**

- **Usage:** Finds documents where the value of a field is not equal to a specified value.
- **Example:** Find all documents where status is not "inactive".

```
db.users.find({ status: { $ne: "inactive" } })
```

Logical Operators

Logical operators combine multiple query conditions.

1. **\$and:**

- **Usage:** Combines multiple conditions; all must be true for a document to match.
- **Example:** Find all documents where age is greater than 25 and status is "active".

```
db.users.find({ $and: [{ age: { $gt: 25 } }, { status: "active" }] })
```

2. **\$or:**

- **Usage:** Combines multiple conditions; at least one must be true for a document to match.
- **Example:** Find all documents where status is "active" or age is less than 20

```
db.users.find({ $or: [{ status: "active" }, { age: { $lt: 20 } }] })
```

3. **\$not:**

- **Usage:** Inverts the effect of a query expression.
- **Example:** Find all documents where status is not "active"

```
db.users.find({ status: { $not: { $eq: "active" } } })
```

4. **\$nor:**

- **Usage:** Combines multiple conditions; none must be true for a document to match.
- **Example:** Find all documents where neither status is "active" nor age is less than 20

```
db.users.find({ $nor: [{ status: "active" }, { age: { $lt: 20 } }] })
```

Array Operators

1. **'\$in'**

- **Usage:** Matches documents where the field's value is in the specified array.
- **Example:** Find all documents where rating is 7,8 or 9".

```
db.bookstore.find({ $rating : { $in : [7,8,9] } })
```

Note : we can do the same thing with 'or' operator :

```
Db.bookstore.find({$or : [{rating:7}, {rating:8}, {rating:9}] })
```

2. '\$nin'

- **Usage:** Matches documents where the field's value is not in the specified array.
- **Example:** Find all documents where status is neither "inactive" nor "banned".

```
db.users.find({ status: { $nin: ["inactive", "banned"] } })
```

3. '\$all':

- **Usage:** Matches arrays that contain all the specified elements.
- **Example:** Find all documents where tags contain both "mongodb" and "database".

```
db.posts.find({ tags: { $all: ["mongodb", "database"] } })
```

Deleting Documents - (Delete)

In MongoDB, documents can be deleted using the `deleteOne()` and `deleteMany()` methods. These methods allow you to delete documents based on specific criteria.

Methods to Delete Documents

1. `deleteOne()`: Deleting single documents

- This method deletes a **single document** that matches the specified filter. If multiple documents match, it deletes only the first one found.
- **Example:** Delete a document where title is "Safe Haven".

```
db.collection.deleteOne({ title: "Safe Haven" })
```

2. `deleteMany()`: Deleting multiple documents

- This method deletes **all documents** that match the specified filter.
- **Example:** Delete all documents where author is "Nicholas Sparks".

```
db.collection.deleteMany({ author: "Nicholas Sparks" })
```

3. Delete All documents

- Use `deleteMany()` to delete entire collection in the DB at once. Use with caution.

- **Command :**

```
db.books.deleteMany({})
```

Important Notes

- **Permanence:** Deleting documents in MongoDB is permanent; there is no "undo" function. Always ensure that your deletion criteria are accurate.
- **Safety:** If you intend to delete documents conditionally, always double-check your query filter to avoid accidentally deleting unintended documents.
- **Best Practice :** its always best to use `_id` field while deleting documents to ensure it deletes the correct instance of the document.
- **Indexing:** Deleting documents does not automatically remove indexes associated with those documents. The index is only updated to exclude the deleted documents.

Update Documents – (Update)

Updating documents in MongoDB is a common operation that allows you to modify existing data in your collections. MongoDB provides several methods to update documents, such as `updateOne()`, `updateMany()`, and `findOneAndUpdate()`. These methods use update operators to specify how documents should be modified.

Methods to Update Documents

1. **`updateOne()`:**

- Updates the **first document** that matches the specified filter. If multiple documents match, only the first document found is updated.

- **Syntax:** `db.collection.updateOne(filter, update, options)`

- **Example:** Update the rating of a book where title is "Safe Haven".

```
db.books.updateOne(  
  { title: "Safe Haven" },  
  { $set: { rating: 9.5 } }  
)
```

- **Explanation:** This sets the rating field to 9.5 for the first document with title "Safe Haven".

2. **updateMany():**

- Updates **all documents** that match the specified filter.
- **Syntax:** `db.collection.updateMany(filter, update, options)`
- **Example:** Update the rating for all books by "Nicholas Sparks".

```
db.books.updateMany(  
  { author: "Nicholas Sparks" },  
  { $set: { rating: 8.5 } }  
)
```

- **Explanation:** This sets the rating field to 8.5 for all documents where the author is "Nicholas Sparks".

3. **findOneAndUpdate():**

- Finds a single document and updates it. This method can also return the original or updated document.
- **Syntax:** `db.collection.findOneAndUpdate(filter, update, options)`
- **Example:** Find a book by title and update its pages.

```
db.books.findOneAndUpdate(  
  { title: "Safe Haven" },  
  { $inc: { pages: 10 } },  
  { returnNewDocument: true }  
)
```

```
    { title: "Safe Haven" },  
    { $inc: { pages: 10 } },  
    { returnDocument: "after" } // Returns the updated document  
  )
```

- **Explanation:** This increments the pages field by 10 and returns the updated document.

Common Update Operators

1. \$set:

- Sets the value of a field in a document.

- **Example:**

```
db.books.updateOne(  
  { title: "Safe Haven" },  
  { $set: { genre: "Romance" } }  
)
```

- **Explanation:** Adds or updates the genre field with the value "Romance".

2. \$unset:

- Removes a field from a document.

- **Example:**

```
db.books.updateOne(  
  { title: "Safe Haven" },  
  { $unset: { rating: "" } }  
)
```

- **Explanation:** Removes the rating field from the document.

3. \$inc:

- Increments the value of a field by a specified amount.

- **Example:**

```
db.books.updateOne(  
  { title: "Safe Haven" },  
  { $inc: { pages: 50 } }  
)
```

- **Explanation:** Increments the pages field by 50.

4. \$push:

- Adds an element to an array.

- **Example:**

```
db.books.updateOne(  
  { title: "Safe Haven" },  
  { $push: { genres: "Adventure" } }  
)
```

- **Explanation:** Adds "Adventure" to the genres array.

5. \$pull:

- Removes an element from an array.

- **Example:**

```
db.books.updateOne(  
  { title: "Safe Haven" },  
  { $pull: { genres: "Adventure" } }  
)
```

- **Explanation:** Removes "Adventure" from the genres array.

6. \$rename:

- Renames a field.

- **Example:**

```
db.books.updateOne(  
  { title: "Safe Haven" },  
  { $rename: { "rating": "bookRating" } }  
)
```

- **Explanation:** Renames the rating field to bookRating.

Examples of Updating Documents

Consider a books collection with the following documents:

```
{ "_id": 1, "title": "Safe Haven", "author": "Nicholas Sparks", "rating": 9.2,  
  "pages": 450 }
```

```
{ "_id": 2, "title": "The Notebook", "author": "Nicholas Sparks", "rating": 8.9,  
  "pages": 300 }
```

```
{ "_id": 3, "title": "To Kill a Mockingbird", "author": "Harper Lee", "rating":  
  9.5, "pages": 400 }
```

Example 1: Update a Single Document

- **Command:**

```
db.books.updateOne(  
  { title: "Safe Haven" },  
  { $set: { rating: 9.8, pages: 460 } }  
)
```

- **Result:** The rating is updated to 9.8 and pages to 460 for the book titled "Safe Haven".

Example 2: Update Multiple Documents

- **Command:**

```
db.books.updateMany(  
  { title: "Safe Haven" },  
  { $set: { rating: 9.8, pages: 460 } }
```

```
{ author: "Nicholas Sparks" },  
  { $inc: { rating: 0.1 } }  
)
```

- **Result:** The rating of all books by "Nicholas Sparks" is incremented by 0.1.

Summary

MongoDB provides flexible and powerful methods to update documents using `updateOne()`, `updateMany()`, and `findOneAndUpdate()`. These methods, combined with various update operators like `$set`, `$unset`, `$inc`, `$push`, and `$pull`, allow you to perform a wide range of updates to your documents, ensuring your data remains accurate and up-to-date.

Indexes

What Are Indexes in MongoDB?

Indexes in MongoDB are special data structures that improve the speed of query operations by reducing the amount of data that needs to be scanned. An index stores a small portion of the collection's data in a form that is easy for MongoDB to search and navigate through quickly, much like an index in a book helps you find topics faster.

By default, MongoDB creates an index on the `_id` field, which is a unique identifier for each document in a collection. You can also create additional indexes on other fields to improve the performance of queries.

How Indexes Work

When you query a collection without an index on the queried field(s), MongoDB must perform a **collection scan** (scan every document in the collection to match the query). With an index, MongoDB can go directly to the relevant documents, skipping over unnecessary ones, which dramatically reduces the time needed for the query.

Types of Indexes in MongoDB

1. **Single-Field Index:** Created on a single field, such as name or age.
2. **Compound Index:** Created on multiple fields, useful when querying on multiple fields in combination.
3. **Multikey Index:** Created on arrays, allowing efficient queries on array elements.
4. **Text Index:** Used to search text within string fields.
5. **Geospatial Index:** Used for location-based data.

Creating an Index: Example

1. Single-Field Index

Let's say you have a users collection and frequently query the age field. Creating an index on age will make these queries faster.

- **Without Index:**

```
db.users.find({ age: 25 }) // This will perform a full collection scan.
```

- **Creating an Index:**

```
db.users.createIndex({ age: 1 }) // 1 means ascending order.
```

- **With Index:** After creating the index, the same query will be faster:

```
db.users.find({ age: 25 }) // This query will now use the index.
```

2. Compound Index

If you frequently query both age and name together, you can create a compound index:

```
db.users.createIndex({ age: 1, name: 1 }) // Index on both 'age' and 'name'.
```

Now, queries that use both fields in the query filter will benefit from this index.

3. Text Index

To perform full-text search on a collection, you can create a text index:

```
db.articles.createIndex({ content: "text" }) // Allows full-text search on 'content' field.
```

- Query Example:

```
db.articles.find({ $text: { $search: "mongodb" } }) // Search for "mongodb" in the content field.
```

When to Use Indexes

1. **Frequent Queries on a Field:** If you query a particular field (like name, age, etc.) very frequently, creating an index on that field makes sense, as it reduces the need for full collection scans.
2. **Sorting and Range Queries:** Indexes speed up sorting operations (.sort()) and range queries (\$gt, \$lt, \$gte, \$lte).
3. **Fields with High Selectivity:** If the field you are indexing has many unique values (e.g., a field like email), an index will significantly improve query performance.
4. **Compound Queries:** If you frequently query multiple fields together, a compound index can help reduce query time.
5. **Large Collections:** If your collection contains a large number of documents, indexes can prevent performance degradation during searches, as collection scans can become too slow.

When Not to Use Indexes

1. **Low Selectivity Fields:** If a field has very few unique values (e.g., status with values like "active", "inactive"), indexing this field may not improve performance. MongoDB will still scan large portions of the collection even with the index.
2. **Write-Heavy Applications:** Indexes can slow down write operations (insert, update, delete), because MongoDB must update the index every time a

document is written. For collections with heavy write operations, avoid unnecessary indexes.

3. **Small Collections:** If a collection contains only a small number of documents, the overhead of maintaining an index may outweigh the performance benefits of using one.
4. **Too Many Indexes:** Indexes take up storage space and slow down writes, so having too many unnecessary indexes can hurt performance. Only index fields that you frequently query.

Example of When to Use Indexes

Assume you have a collection `products` with the following documents:

```
{ _id: 1, name: "Laptop", price: 1500, category: "Electronics" }
```

```
{ _id: 2, name: "Phone", price: 800, category: "Electronics" }
```

```
{ _id: 3, name: "Shoes", price: 100, category: "Fashion" }
```

```
{ _id: 4, name: "T-Shirt", price: 30, category: "Fashion" }
```

1. If you frequently search for products by price (e.g., `db.products.find({ price: 800 })`), you should create an index on price:

```
db.products.createIndex({ price: 1 });
```

2. If you frequently search for products by category and price together, you could create a compound index:

```
db.products.createIndex({ category: 1, price: 1 });
```

Example of When Not to Use Indexes

1. If your application is write-heavy, and you are constantly inserting and updating products, then creating many indexes might slow down performance due to the overhead of updating the indexes.
 2. If the category field only contains a few possible values like "Electronics", "Fashion", etc., and you query on it, an index may not help much because it doesn't reduce the search space by a large amount.
-

Summary

- **Indexes** speed up query performance by reducing the number of documents MongoDB needs to scan.
- **Use indexes** when you frequently query specific fields, perform sorting, or have large collections with high selectivity.
- **Avoid indexes** on fields with low selectivity, in write-heavy applications, or on small collections where the overhead is not justified.